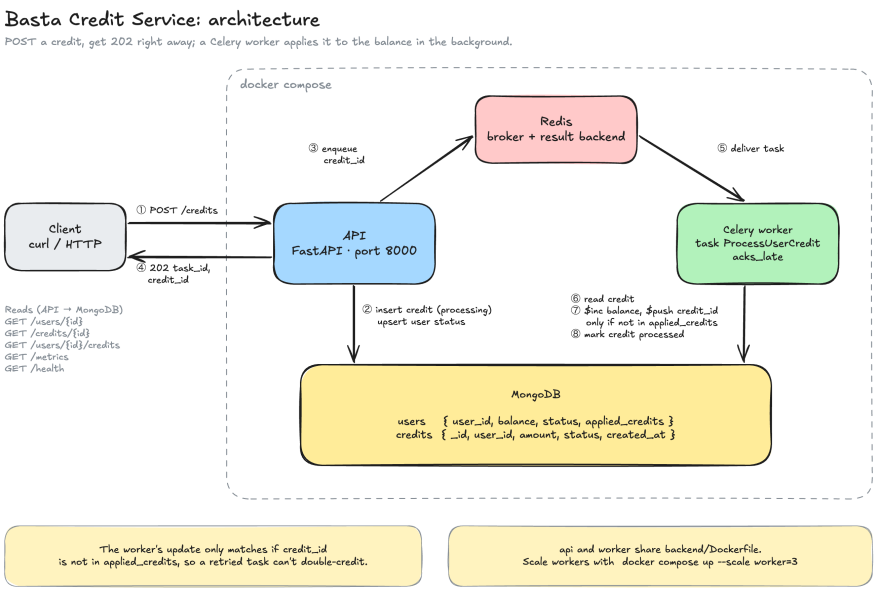
Eseosa Asiruwa · Repo: github.com/easiruwa/basta-credit-service

Overview

A client asks the API to add credit to a user. The API records the request and returns `202 Accepted` right away. A Celery worker then applies the credit to the user's balance in the background. Both the user and each credit carry a `status` (`processing` → `processed`), so a client can see where a credit is at any point.

Architecture

Components and the order of calls for one credit →



How a credit request moves through the flow

1. `POST /credits {user_id, amount}`. FastAPI rejects a bad body with `422` (amount must be > 0, user_id can't be empty).
2. The API inserts the credit as `processing` and upserts the user with `status: processing`. It never touches `balance`. If a user wasn't previously in the collection, they are added.
3. The API queues the task with only the `credit_id`, and immediately returns `202` as a response with `task_id` and `credit_id`.
4. The worker reads the credit from Mongo, adds the amount to the user's balance in one atomic update, then marks the credit `processed`.

The queue carries an id, not an amount. The worker reads everything it needs from Mongo, so there is one copy of the data and nothing in Redis can drift from it.

Key design decisions

I added one document per credit request so every balance change has a record behind it. It gives each credit its own status, feeds the credit history endpoint, and is what `/metrics` counts.

Additionally, the task runs with `acks_late`, so if a worker dies mid-task Celery delivers it again. Without a guard, that retry would credit the user twice. The worker's update only matches if the credit isn't already recorded on the user:

Mongo applies an update to one document atomically, so the check and the balance change can't come apart. A repeat matches nothing, logs a warning, and just marks the credit `processed`. I considered two alternatives. Marking the credit as taken first and then applying it loses the credit if the worker crashes in between. A multi-document transaction is the usual production answer, but it needs a replica set and the local setup runs a single Mongo. The cost of this approach: `applied_credits` grows with every credit and would eventually hit Mongo's 16 MB document limit for a very busy user.

I added `$inc` inside Mongo in one step, so two workers crediting the same user can't overwrite each other (a read, add, write would). It also creates `balance` on the first credit. An early version had the API set `balance: 0` on every request, which wiped existing balances. Now the API only sets `status`, and `created_at` goes through `$setOnInsert`.

`task_id` is the Celery job, which the spec asks for. `credit_id` is what a client uses to check on the credit later with `GET /credits/{credit_id}`.

For metrics (and later potential alerting), Celery runs tasks in several processes, so in-memory counters would each hold part of the count and reset on restart. One aggregation returns counts by status plus the age of the oldest `processing` credit. That age is the number to alert on: if it keeps growing, the worker is down or falling behind.

Implementation approach

- The `Status` enum and models live in `models.py`, so the API and worker import them without importing each other. Settings come from environment variables via `pydantic-settings`.
- One Docker image started twice by Docker Compose, once as the API and once as the worker. Each restarts on its own, logs stay separate, and `docker compose up --scale worker=3` adds workers. Mongo, Redis and the API have health checks, and containers don't run as root.
- `make setup / up / test / check` cover the common tasks. `ruff` formats and lints through a pre-commit hook. CI runs lint, the tests against a real Mongo, and a Docker build on every push and PR.

Testing strategy

The important logic lives in the Mongo updates (`$inc`, `$setOnInsert`, upserts, the aggregation), so the tests run against a real MongoDB with its own `credit_test` database, emptied around each test. A fake database wouldn't show whether those updates are right. Celery and Redis aren't needed because API tests replace `.delay()` with a fake that records what was queued, and worker tests call the task function directly.

Trade-offs and next steps

- Production should store whole cents or `Decimal128`.
- If Redis is down after the credit is saved, it stays `processing`. We could fix this with a job that re-queues stale `processing` credits.
- The `applied_credits` guard stops the same credit from being applied twice, but each POST gets a new credit_id. A client that times out and resends creates a second, separate credit. An Idempotency-Key header, stored on the credit with a unique index, would let the API return the original instead.
- `max_retries` is set but nothing triggers a retry or sets `failed` yet. The next step would to implement `autoretry_for` with backoff, then mark the credit `failed`.
- When we have a massive set of credits, it can become slow to query the collection. To solve this, we can add indexes on `users.user_id` (unique), `credits.user_id` and `credits.status`, and cursor pagination on credit history.